Sensores, Conversão AD/DA e Interrupções

Tecnologias de Comunicação para a Internet das Coisas, Aula 02

Catarina Silva

ESTGA, Universidade de Aveiro | 2026

Agenda

- Sensores: do mundo analógico ao mundo digital
- Conversão analógico-digital: amostragem e resolução
- Interrupções por hardware e por software
- O equipamento da sessão: Arduino Uno e os três sensores do guião
- Programação visual: um primeiro contacto com o Node-RED
- Estrutura da sessão prática (2h)

Da Teoria à Prática

- Na Aula 01 vimos a arquitetura de referência de IoT e o paradigma M2M: sensores captam grandezas do mundo real e alimentam uma cadeia de comunicação até à *cloud*.
- Nesta aula fecha-se o primeiro elo dessa cadeia: como um microcontrolador lê um sensor e como reage a eventos sem permanecer ocupado à espera deles.
- Estes 45 minutos preparam diretamente os sete exercícios do guião prático desta aula.

O que é um Sensor (1/2)

- Um sensor converte uma grandeza física (temperatura, humidade, distância, luz, ...) num sinal elétrico que um sistema digital consegue processar.
- Pode ser simples, dedicado a uma única grandeza, ou complexo, combinando várias leituras e algum pré-processamento.
- É o termo genérico por detrás daquilo a que chamamos informalmente "as coisas" de um sistema IoT.

O que é um Sensor (2/2)

- **Sensores digitais**: já entregam uma leitura discreta ou um protocolo próprio (ex.: o DHT11, que devolve valores por um barramento de um fio).
- **Sensores analógicos**: entregam uma tensão contínua, proporcional à grandeza medida (ex.: o sensor de humidade do solo, lido no pino `A0`).
- Os três sensores do guião cobrem exatamente estes dois casos, mais um sensor por tempo de eco (HC-SR04).

Porque Converter Analógico em Digital (1/2)

- Os microprocessadores são máquinas digitais: só manipulam sequências de bits.
- O mundo físico é essencialmente analógico: as grandezas variam de forma contínua no tempo.
- É necessário um circuito de interface entre os dois mundos.

Porque Converter Analógico em Digital (2/2)

- O **conversor analógico-digital** (ADC) transforma a tensão contínua do sensor numa sequência de números que o microcontrolador pode processar.
- O **conversor digital-analógico** (DAC) faz o caminho inverso, quando é o sistema digital a gerar um sinal contínuo.
- No guião utiliza-se apenas o ADC: é ele que está por detrás da função `analogRead()`.

Amostragem (1/2)

- A amostragem consiste em medir o sinal de entrada em instantes periódicos, e não continuamente.
- O conjunto de valores medidos é apenas uma representação do sinal original, nunca o sinal exato.
- Pergunta central: com que frequência é preciso medir para não perder informação relevante?

Amostragem (2/2)

- O **Teorema de Nyquist** responde: a frequência de amostragem f_s tem de ser pelo menos o dobro da componente de frequência mais rápida do sinal ($f_s = 2B$).
- Na prática usa-se uma margem confortável, tipicamente 5 a 10 vezes B.
- Para as grandezas físicas lentas deste guião (temperatura, humidade, distância), uma leitura por segundo é largamente suficiente.

Resolução e Erro de Quantificação

- Além de *quando* medir, importa *com que precisão*: a resolução é o número de bits usados para representar cada amostra.
- Mais bits permitem mais valores discretos e menor erro de quantificação, à custa de mais memória e de mais tempo de conversão.
- Exemplo: 3 bits distinguem 8 níveis; 4 bits distinguem 16, ou seja, o dobro da precisão.

Na Prática: o ADC do Arduino Uno

- O Arduino Uno tem um ADC de 10 bits: cada leitura de `analogRead()` devolve um valor entre 0 e 1023.
- Esse valor é proporcional à tensão lida no pino analógico, entre 0V e a tensão de referência (5V, por omissão).
- É este o mecanismo utilizado a seguir para ler o sensor de humidade do solo.

Porquê Interrupções?

- Até agora, o `loop()` do Arduino verifica o estado das coisas repetidamente: é o chamado *polling*.
- A sondagem funciona, mas desperdiça ciclos de processamento e pode não detetar um evento muito curto ocorrido entre duas verificações.
- Uma **interrupção** inverte a lógica: o processador continua a sua tarefa e só é desviado quando o evento realmente acontece.

Interrupções por Hardware (1/2)

- Certos pinos digitais do Arduino Uno (2 e 3) estão ligados diretamente a um controlador de interrupções do microcontrolador.
- Uma mudança elétrica nesse pino, por exemplo ao premir um botão, dispara de imediato uma **ISR** (*Interrupt Service Routine*), independentemente do que o `loop()` estivesse a fazer.
- É o mecanismo mais direto para reagir a eventos externos rápidos: um botão, um sensor de fim-de-curso, um pulso de outro circuito.

Interrupções por Hardware (2/2)

- Em código: `attachInterrupt(pino, funcaoISR, modo)`, em que o modo define quando disparar (`CHANGE`, `RISING`, `FALLING`).
- No exercício correspondente, um botão ligado ao pino 2 dispara uma ISR que acende ou apaga o LED, sem que o `loop()` verifique coisa alguma.
- Note-se que o `loop()` do exemplo fica praticamente vazio: todo o trabalho ocorre na ISR.

Interrupções por Software (1/2)

- Nem todos os eventos vêm de um pino: por vezes queremos apenas que algo aconteça **periodicamente**, de tempos a tempos.
- É o caso de um temporizador: em vez de contar manualmente o tempo no `loop()`, um periférico interno do microcontrolador conta por nós e dispara uma interrupção quando o tempo definido se esgota.
- No Arduino Uno isto é acessível através da biblioteca `TimerOne`.

Interrupções por Software (2/2)

- Em código: `Timer1.initialize(us)` define o período (em microssegundos) e `Timer1.attachInterrupt(funcao)` associa a ISR a esse temporizador.
- No exercício correspondente, uma ISR periódica de 10 em 10 segundos atua sobre um contador que o `loop()` também incrementa, exemplo elementar de duas linhas de execução a partilhar a mesma variável.
- Este exercício prepara diretamente o terreno para as leituras periódicas de sensores que se seguem.

Boas Práticas numa ISR

- Uma ISR deve ser **curta e rápida**: enquanto executa, o resto do programa espera.
- Variáveis partilhadas entre a ISR e o `loop()` devem ser declaradas `volatile`, para o compilador não as guardar apenas em registos e assumir valores desatualizados.
- Devem evitar-se `delay()` e operações lentas dentro de uma ISR, erro mais comum de quem começa a usar interrupções.

O Equipamento da Sessão (1/2)

- **Arduino Uno**: plataforma aberta de microcontrolador, com hardware e software licenciados sob GPL/LGPL, base de todos os exercícios.
- **LED onboard (pino 13) e botão de pressão**: para os dois exercícios de interrupções.
- **Sensor de humidade do solo (Funduino)**: sensor resistivo analógico, lido em `A0`.

O Equipamento da Sessão (2/2)

- **DHT11**: sensor digital de temperatura e humidade, com protocolo próprio de um único fio, razão pela qual exige uma biblioteca dedicada (`dht.h`).
- **HC-SR04**: sensor ultrassónico de distância; mede o tempo de eco de um pulso sonoro para calcular a distância a um obstáculo.
- O guião trabalha apenas com este conjunto, que é o disponível em laboratório.

Cuidados de Montagem

- Os circuitos devem ser montados com o Arduino **desligado do USB**, ou pelo menos sem programa em execução que ative os pinos envolvidos.
- A polaridade dos sensores deve ser confirmada antes de alimentar: VCC e GND trocados podem danificar o componente.
- Não é exigida experiência prévia de eletrónica: cada exercício do guião indica exatamente que fios ligar a que pinos.

Programação Visual: No-Code e Low-Code

- Nem toda a lógica de um sistema IoT precisa de ser escrita linha a linha: ferramentas **no-code** (sem código) e **low-code** (pouco código) permitem montar comportamentos ligando blocos visuais.
- Não substituem a programação, uma vez que o Arduino continua a necessitar do seu programa, mas aceleram a prototipagem do que vem a seguir ao sensor: painéis, regras e integrações.
- É uma abordagem particularmente comum em IoT, onde é preciso ligar muitas peças diferentes (sensores, protocolos, bases de dados, notificações) rapidamente.

Node-RED

- O **Node-RED** é uma ferramenta open-source de programação visual baseada em *flows*: arrastam-se e ligam-se nós num navegador, sem escrever JavaScript.
- Cada nó representa uma pequena função, como ler uma porta série, comunicar por MQTT ou desenhar um gráfico, e os fios entre eles representam o percurso dos dados.
- Na sessão laboratorial o Node-RED é a ferramenta principal: com o firmware `StandardFirmata` carregado na placa, os pinos do Arduino passam a ser lidos e escritos diretamente a partir dos nós, sem escrita de código.
- Apenas os sensores com protocolos temporizados (DHT11 e HC-SR04) exigem um *sketch* próprio, fornecido pronto a carregar.
- Voltaremos ao Node-RED na Aula 12, desta vez a receber dados por MQTT em vez de porta série.

Estrutura da Sessão Prática (2h)

- Sete exercícios, todos com montagem física: LED, botão, sensor de humidade do solo, DHT11, HC-SR04 e um painel final que integra as várias leituras.
- Os três primeiros fazem-se inteiramente em Node-RED, sem escrever código; nos restantes, os *sketches* necessários são fornecidos.
- O exercício 6 volta ao nível do microcontrolador, para comparar o modelo orientado a eventos dos fluxos visuais com as interrupções por hardware e por software.

Síntese da Aula

- Sensores fazem a ponte entre o mundo analógico e o sistema digital; o ADC do Arduino Uno amostra essa grandeza em 10 bits, valores entre 0 e 1023.
- As interrupções, por hardware ou por software, permitem reagir a eventos sem desperdiçar ciclos em sondagem, desde que as ISRs sejam curtas e as variáveis partilhadas sejam declaradas `volatile`.
- As ferramentas de baixo código, como o Node-RED, permitem adquirir e visualizar esses dados sem escrita de código, desde que o protocolo do sensor o permita, tema retomado com MQTT na Aula 12.

Obrigada.

Questões e comentários